

Introduction. My research interests lie at the intersection of human-computer interaction (HCI) and generative AI, specifically in the domain of software engineering, where AI is materially altering common tools and practices. Most ‘Big Tech’ companies have even internally adopted an *AI-first* philosophy, aiming to integrate AI in all aspects of their products. On the other hand, a wave of public opinion and research has criticized the integration of AI for programming, demonstrating that AI may not provide practical benefits to programmers, and that relying on AI may in fact harm them in the long term. This perspective thus advocates for a *programmer-first* approach to AI. With my varied research experiences in both academia [1, 2, 3] and industry [4, 5, 6] across human-AI interactions, developer tools, and computing education, I believe I am in a unique position to reconcile both these streams of thought. Specifically, my research has gravitated towards a **collaboration-first** paradigm for AI – developing technology that prioritizes effective and deliberate collaborations over both programmer and AI, facilitating learning and growth for programmers. As a PhD student and beyond, I intend to discover fundamental design principles for this paradigm, both by evaluating current tools to understand their limitations as well as by building new technology that addresses these limitations through collaboration.

Human-Centered Biases My first foray into human-centered research was a literature review paper that investigated cultural biases in end-user programming research, arguing to build tools for all rather than only for Western societies (VL/HCC 2023, [link](#)) [1]. Under the guidance of Prof. Sruti Ragavan, I developed fundamental research skills in HCI through this work, such as conducting literature reviews, critically analyzing research papers as well as thematic and quantitative analysis. The perspective of inclusivity has stayed with me – I recently wrote a short paper investigating similar biases in next-word recommendations by LLMs (AAAI 2026, [link](#)) [7]. This perspective was reinforced during my junior year as an undergraduate, when I was nominated for a semester exchange program at EPFL (École Polytechnique Fédérale de Lausanne). There, I was selected for a semester project at the [ML4ED Lab](#), working on intersectional biases in demographic-based oversampling algorithms. This experience was my first exposure to machine learning (ML) research, and it taught me the importance of evaluating ML and AI tools based on how well they capture the unique needs of the human using them. To be truly useful, AI tools should thus collaborate with programmers to understand their specific context.

AI for Programming Education. During my senior year, I investigated the challenges and opportunities that AI presents for CS students, and by extension, for novice programmers. As part of a cross-university research group, I helped design a survey that explored student trust in AI tools for CS education (SIGCSE 2024, [link](#)) [2]. This work revealed that CS students have various conflicting opinions on the usefulness and risks of AI. To ground these findings in real-world behavior, I then led my own within-subject comparative user study investigating how CS2 students learn programming concepts with and without LLMs. This was my first experience conducting end-to-end research, from developing research questions and study designs to then conducting the study, performing mixed-methods data analysis and formalizing the insights in a research paper (under review, [link](#)) [3]. In this study, we found that CS students performed worse in a theory-based quiz after learning with LLMs as compared to learning with web search. Since LLMs are designed to provide direct solutions to users’ queries and tasks (an *AI-first* approach), they may fail to provide novices with the same support as human-curated educational websites for programming that prioritize learners’ needs (a *programmer-first* approach). To understand novice programmers’ needs and support long-term growth, AI tools must be fundamentally redesigned to collaboratively work *with* novice programmers, rather than *for* them.

Research at PROSE. I am currently working full time as a pre-doctoral research fellow at the [PROSE team](#) in Microsoft, led by Dr. Sumit Gulwani. Here, I have been working on understanding and building novel human-AI interactions for programmers and end users, particularly in the context of collaborations with AI agents (‘agent’ refers to an AI tool that can make direct edits to codebases and iterate on its own output). This fellowship has provided me with the opportunity to build my technical skills and refine my research interests by working on state-of-the-art AI tools for widely used products such as Visual Studio Code and Excel. The rapid pace of AI development in industry has also motivated me to think more critically about the practical applications and the longevity of my research.

During my first few months at PROSE, I worked on open-ended research questions around the integration of agentic AI programming assistants into IDEs. While the prevalent research narrative around such tools

focused on increasing their performance on automated benchmarks, I instead applied a human-centered approach to find more pertinent problems – specifically, I found that AI tools seemed to rely on either the AI or the user for different parts of the software development process (such as the dependency on the programmer for providing requirements and on the AI for writing code), which can introduce unnecessary bottlenecks to effective collaboration. For instance, I conducted a user study investigating the usefulness of TEN, an LLM-based tool that reconstructed tables from images (under review, [link](#)) [4]. While the best way to reconstruct a table using an LLM is to organize the table based on header columns and rows, this workflow relies on the end user programmer to assess the accuracy of the data in these tables, a task that they often found tedious. Building AI with the perspective of collaborative goals, rather than the best autonomous performance, might lead to more effective workflows.

Towards creating such collaborative experiences, I worked on building TableTalk (TOCHI 2025, [link](#)) [5], an agentic tool to support end user programmers in creating new tables with Microsoft Excel. Rather than forcing users to manually adapt Excel templates (*programmer-first*) or generating a boilerplate workbook based on a single prompt (*AI-first*), TableTalk collaborates with users by requesting feedback at different stages of table construction to build the ideal workbook for their particular context and needs. Excel provided a great playground to study human-AI collaborations – as spreadsheet tasks can be both technically complex as well as ambiguous and context-dependent, effective collaborations are all the more essential. To understand the needs of end user programmers, I first conducted open coding and qualitative analysis for a formative user study and a spreadsheet dataset. We distilled the insights from this analysis into specific design goals. I then worked on technically implementing these goals into an agentic LLM-based backend and an interactive interface for TableTalk. Following this, we conducted an evaluation study, finding that TableTalk produced higher quality tables than a non-collaborative baseline. Remarkably, this higher quality did not come at the cost of higher effort – despite the fact that TableTalk regularly required user input, participants reported a lower cognitive load when working with TableTalk as compared to the baseline. This insight demonstrates the potential of the collaboration-first approach as not only being empowering in the long term, but further providing short-term benefits in task outcomes and usability.

Despite all the benefits of TableTalk, the agentic paradigm for AI still presented difficulties for effective collaborations. For example, in the same evaluation study, participants reported mixed feelings about the proactivity of the agent, noting that it reduced their sense of control over the AI. To explore such issues further, I led a project on investigating collaborations between professional programmers and a state-of-the-art agentic IDE (ASE 2025, [link](#)) [6]. Under the guidance of [Dr. Emerson Murphy-Hill](#), I created the study design and protocol for an exploratory task-based user study that enabled us to observe how programmers use an AI agent in a highly ecologically valid setting. I then conducted in-depth qualitative coding and analysis by building and applying two independent codebooks on participants' actions and their interactions with the agent. My analysis found that such collaborations are indeed ripe with challenges. Programmers either overrelied on the agent (the *AI-first* paradigm, often leading to failure) or took matters largely into their own hands, not giving the agent any agency at all (the *programmer-first* paradigm, often leading to slow progress). To leverage the strength of both AI and programmer, AI tools must engage in critical discussions with programmers, or in other words, must prioritize collaboration over immediate task completion and obeying the user.

My findings across research projects have thus echoed the need for collaboration-first AI, specifically arguing to design AI tools with deliberate friction, a respect for programmer agency, and goals beyond short-term task completion. These projects have equipped me with the research skills, experience, and most critically the research directions to pursue a PhD by building on my empirical findings to explore the fundamental principles of collaboration-first AI for programmers. Looking beyond my PhD, I aim to continue working in an academic environment as a postdoc and eventually as a professor to explore how we can make AI useful across all programming communities.

Why $\langle \text{Department} \rangle$ at $\langle \text{University} \rangle$? The vibrant interdisciplinary research community in the $\langle \text{Department} \rangle$ at $\langle \text{University} \rangle$ would provide the ideal environment for me to pursue my research goals. **Prof. X's** research interests in ... perfectly match my experiences and interests in **Prof. Y's** aims of ... fit well with my Further, the research interests of **Prof. Z** in ... strongly align with my goals of

References

- [1] Harshit Goel, **Aayush Kumar**, and Sruti Srinivasa Ragavan. End-User Programming is WEIRD: How, Why and What to Do About It. In *Proc. of VL/HCC 2023*.
- [2] Matin Amoozadeh, David Daniels, Daye Nam, **Aayush Kumar**, Stella Chen, Michael Hilton, Sruti Srinivasa Ragavan, and Mohammad Amin Alipour. Trust in Generative AI among Students: An exploratory study. In *Proc. of SIGCSE 2024*.
- [3] **Aayush Kumar**, Daniel Prol, Amin Alipour, and Sruti Srinivasa Ragavan. To Google or To ChatGPT? A Comparison of CS2 Students' Information Gathering Approaches and Outcomes, 2025. Under review.
- [4] Nikita Mehrotra, **Aayush Kumar**, Sumit Gulwani, Arjun Radhakrishna, and Ashish Tiwari. TEN: Table Explicitization, Neurosymbolically. Under Review.
- [5] Jenny T. Liang, **Aayush Kumar**, Yasharth Bajpai, Sumit Gulwani, Vu Le, Chris Parnin, Arjun Radhakrishna, Ashish Tiwari, Emerson Murphy-Hill, and Gustavo Soares. TableTalk: Scaffolding Spreadsheet Development with a Language Agent. *ACM Trans. Comput.-Hum. Interact.*
- [6] **Aayush Kumar**, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild. In *Proc. of ASE 2025*.
- [7] **Aayush Kumar** and Sanket Mhatre. UnWEIRDing LLM Entity Recommendations. In *Proc. of 2nd Workshop on Language Models for Underserved Communities (LM4UC), AAAI 2026*. To appear.